

AI Models For Chess

Abstract

We compare four decision frameworks—Monte-Carlo Tree Search (MCTS), Minimax, convolutional neural networks (CNNs), and Long-Short Term Memory (LSTM) sequence models—within a common chess test-bed. Under a tight per-move time budget, MCTS explores tens of thousands of positions at a consistent four-ply horizon and remains competitive against a strong reference engine. Minimax, in contrast, plays complete games but rarely converts balanced positions into wins, revealing the limits of a purely material evaluation. A CNN trained on expert moves reproduces opening theory convincingly yet degrades as the position drifts beyond its training manifold. The LSTM model captures temporal patterns in master play, predicting the next move far above chance and thus offering a probabilistic prior for search. Taken together, the results suggest a hybrid engine: a CNN-guided MCTS for the opening and middlegame, a table-driven Minimax for simplified endings, and an LSTM opponent model to bias move selection throughout. Ongoing work focuses on integrating these components within a single real-time system and on strategies that stabilise learning when the models are trained jointly.

Introduction

Chess has been regarded as a challenge for artificial intelligence, offering an environment for exploring strategic decision-making, planning, and prediction. Its fully observable, deterministic nature, coupled with an enormous but well-defined state space, makes it an ideal testbed for developing and evaluating AI models. Unlike many real-world tasks, chess provides clear rules, measurable outcomes, and a competitive setting that pushes agents to make high-quality decisions under pressure.

Historically, chess has mirrored milestones in AI development. From early symbolic systems and handcrafted evaluation functions to the landmark victory of IBM's Deep Blue over Garry Kasparov in 1997, each advancement reflected a shift in how machines were designed to reason. More recently, systems like DeepMind's AlphaZero have redefined the state-of-the-art by combining deep reinforcement learning with Monte Carlo Tree Search, achieving superhuman performance through self-play and general-purpose learning.

In this project, we explore how different classes of AI models—ranging from classical algorithms to modern neural architectures—can be applied to the task of playing chess. Our primary objective is to understand the distinct mechanisms by which these models reason, learn, and make decisions. Specifically, we implement and analyze four approaches: Minimax with Alpha-Beta Pruning, Monte Carlo Tree Search (MCTS), Convolutional Neural Networks (CNNs), and Long Short-Term Memory (LSTM) networks.

Rather than optimizing a single model for peak performance, we aim to compare their behaviors across key chess situations—openings, middlegames, and endgames—to reveal their respective strengths and limitations.

Background

Playing chess can be formally defined as a sequential decision-making problem under perfect information. At each turn, an agent is presented with a fully observable state (the board configuration) and must choose a legal action (a move) that leads to a new state. The objective is to select a sequence of actions that culminates in a favorable terminal state—ideally, a win. This setting maps naturally to classical AI formulations such as search problems and Markov decision processes (MDPs), although the state space in chess is far too large for exhaustive enumeration.

Symbolic Methods

Classical AI approaches to chess rely on symbolic search algorithms that simulate potential move sequences. One foundational method is the **Minimax** algorithm, which evaluates the game tree assuming both players act optimally. The agent selects a move that maximizes its worst-case outcome. Due to the exponential growth of the game tree, **Alpha-Beta Pruning** is used to discard branches that cannot influence the final decision, enabling deeper search within fixed computational budgets. These methods require a static evaluation function to score non-terminal positions based on heuristics like material count, mobility, and king safety.

Simulation-Based Search

Monte Carlo Tree Search (MCTS) offers a probabilistic alternative to deterministic search. Rather than evaluating all branches, MCTS selectively expands promising ones based

on outcomes of simulated playouts. Over multiple iterations, it builds a partial tree, using a balance of exploitation and exploration (typically guided by the Upper Confidence Bound formula). MCTS does not require a handcrafted evaluation function and adapts well to unfamiliar or dynamic positions, making it a strong candidate for mid- and endgame play.

Deep Learning Methods

In contrast to symbolic reasoning, **neural networks** learn representations directly from data. **Convolutional Neural Networks (CNNs)** are particularly effective for spatially structured inputs like chess boards. By treating the board as a multi-channel 8x8 image, CNNs can identify tactical and positional patterns (e.g., pins, forks, or castling formations). Meanwhile, **Recurrent Neural Networks (RNNs)**—especially **Long Short-Term Memory (LSTM)** networks—are designed to capture temporal dependencies. When applied to chess, LSTMs can track sequences of moves to model strategic progression or predict opponent behavior.

Together, these methods represent a diverse toolkit for artificial agents. Symbolic algorithms provide transparency and control; simulation-based methods offer adaptive planning; and neural networks bring flexibility and learning from experience.

Related Work

A wide variety of AI methods have been applied to chess, each reflecting broader trends in the evolution of artificial intelligence.

AlphaZero and Reinforcement Learning

One of the most significant advances in recent years is **AlphaZero**, developed by DeepMind. AlphaZero uses deep reinforcement learning combined with MCTS, training purely via self-play without human data. It replaces handcrafted evaluation functions with a deep residual CNN that predicts both the policy (likely next moves) and value (expected outcome) from a given position. The system demonstrated superhuman performance in chess, shogi, and Go. However, implementing such a system requires massive computational resources and fine-tuned reinforcement learning infrastructure, making it impractical for small-scale academic projects.

Supervised Learning Approaches

Several systems have used **supervised learning** to predict moves based on expert data. One notable example is **Maia**, a CNN trained on Lichess games across different Elo ratings. Maia learns to emulate human decision-making at various skill levels and has been used to study human-like behavior in AI. While Maia excels at human move prediction, it does not optimize for winning games and lacks planning components like tree search.

Policy Gradient and TD Learning

Temporal Difference (TD) methods and **policy gradient** approaches have also been explored in chess. These methods

involve learning value functions or policies through trial-and-error, adjusting weights to maximize expected return. While promising, such reinforcement-based approaches are sensitive to reward shaping and may suffer from instability without extensive tuning—especially in games with sparse rewards like chess.

Positioning Our Work

Rather than replicating state-of-the-art systems, our project aims to analyze and compare four representative models that reflect key categories of AI design: symbolic reasoning (Minimax), simulation-based planning (MCTS), spatial learning (CNN), and temporal pattern modeling (LSTM). By implementing and testing each model, we evaluate how they reason, learn, and perform across different phases of the game. By directly comparing these four approaches, we can pinpoint where each method improves or struggles—whether in the opening, middlegame tactics, or endgame technique—and use those lessons to guide the design of even stronger chess engines.

Methods

Monte Carlo Tree Search Implementation

MCTS repeatedly performs four steps from the current position:

1. **Selection.** Starting at the root, descend the tree by picking child moves that balance *exploitation* (high observed win rate) and *exploration* (moves tried fewer times). We also apply a small *repetition penalty* so that paths which repeat the same position too often are slightly deprioritized.
2. **Expansion.** When we reach a node with untried legal moves, we choose one at random (or with a simple heuristic), add it as a new child, and step into it.
3. **Rollout (Simulation).** From the new node, play a short random game (up to a fixed depth). During this phase we count every position we examine, and at the end we evaluate the final board with a very fast material + mobility estimator.
4. **Backpropagation.** We take the result (win = 1, draw = 0.5, loss = 0) and update the *visit* and *win* counts of every node along the path we followed.

After thousands of these iterations, the child of the root with the highest visit count is chosen as the move to play.

Equations In our MCTS implementation we maintain, for each position s and move a :

$$N(s, a) = \#\{\text{simulations that passed through } (s, a)\},$$

$$W(s, a) = \sum_{\text{those simulations}} z.$$

where

$$z = \begin{cases} 1, & \text{if the root player eventually won,} \\ 0.5, & \text{if the game was a draw,} \\ 0, & \text{if the root player lost.} \end{cases}$$

We then set

$$Q(s, a) = \frac{W(s, a)}{N(s, a)}, \quad (1)$$

$$N(s) = \sum_{b \in \mathcal{A}(s)} N(s, b). \quad (2)$$

At selection time we pick

$$a^* = \arg \max_{a \in \mathcal{A}(s)} \left[Q(s, a) / (1 + \beta R(s, a)) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right],$$

where the first term is exploitation (down-weighted by repetition $R(s, a)$), and the second term is the usual UCT exploration bonus.

Algorithm 1 MCTS with Repetition Penalty and Fast Rollout

```

1: function MCTS_SEARCH( $s_0, M, c, \beta, d_{\max}$ )
2:   create root node  $n_0$  with fields:
      $n_0.s \leftarrow s_0, n_0.N \leftarrow 0, n_0.W \leftarrow 0, n_0.R \leftarrow 0,$ 
      $n_0.untried \leftarrow \mathcal{A}(s_0)$ 
3:   for  $i \leftarrow 1$  to  $M$  do
4:      $n \leftarrow n_0, \pi \leftarrow []$  ▷ selection path
5:     while  $n.untried = \emptyset$  and  $n.s$  not terminal do
6:       choose child

        $n' = \arg \max_{m \in \text{children}(n)} \left[ \frac{W_{n,m}}{N_{n,m} (1 + \beta R_{n,m})} + c \sqrt{\frac{\ln N_n}{N_{n,m}}} \right]$ 

7:       append  $n'$  to  $\pi, n \leftarrow n'$ 
8:     end while
9:     if  $n.s$  not terminal and  $n.untried \neq \emptyset$  then
10:      pick and remove move  $a$  from  $n.untried$ 
11:       $s' \leftarrow \text{Step}(n.s, a)$ 
12:      create child  $n'$  with:
          $n'.s \leftarrow s', n'.N \leftarrow 0, n'.W \leftarrow 0, n'.R \leftarrow n.R + [n.s = s']$ 
13:      add  $n'$  to  $n.children$ , append  $n'$  to  $\pi, n \leftarrow n'$ 
14:    end if ▷ rollout from  $n.s$ 
15:     $z \leftarrow \text{ROLLOUT}((n.s, d_{\max}))$  ▷ simulation
    result for root side
16:    for all  $n \in \pi$  in reverse order do
17:       $n.N \leftarrow n.N + 1$ 
18:       $n.W \leftarrow n.W + z$ 
19:       $z \leftarrow 1 - z$ 
20:    end for
21:  end for
22:  return move  $a^* = \arg \max_{a \in \mathcal{A}(s_0)} N(n_0 \xrightarrow{a})$ 
23: end function

```

Algorithm 2 Rollout Policy

```

1: function ROLLOUT( $s, d_{\max}$ )
2:   depth  $\leftarrow 0$ 
3:   while  $s$  not terminal and depth  $< d_{\max}$  do
4:     pick a move  $a$  uniformly at random from  $\mathcal{A}(s)$ 
5:      $s \leftarrow \text{Step}(s, a)$ 
6:     depth  $\leftarrow$  depth + 1
7:   end while
8:   if  $s$  is checkmate then
9:     return 1 if root side to move lost, else 0
10:  else if  $s$  is draw by stalemate or insufficient material
    then
11:    return 0.5
12:  else
13:    return cheap_evaluate( $s$ ) ▷ e.g.
    material+mobility
14:  end if
15: end function

```

Rollout Policy and Evaluation Each simulation (rollout) from a given position proceeds as follows:

1. Play up to $d_{\max}$ moves by choosing uniformly random legal moves. We also increment our `positions_analyzed` counter for each position visited.
2. If the simulation reaches a terminal result:

$$z = \begin{cases} 1.0, & \text{if our side wins,} \\ 0.5, & \text{if the game is drawn,} \\ 0.0, & \text{if our side loses.} \end{cases}$$

Return z immediately.

3. Otherwise (no checkmate or draw within $d_{\max}$ moves), compute a quick heuristic evaluation:

$$\text{value} = 0.5 + 0.5 (\text{material_score} + \text{mobility_score}),$$

$$\text{value} \in [0, 1].$$

Here, `material_score` and `mobility_score` are normalized sums of piece-count and legal-move counts. Return this value.

To avoid recomputing these cheap evaluations, we store them in a small LRU cache keyed by the full FEN string. If the same position appears again, we look up the value in $O(1)$ time instead of recalculating.

Complexity and performance. A single move requires roughly $M(h + d)$ board updates, where M is the number of simulations, $h \approx 15\text{--}30$ is the height of the selected tree path, and $d = d_{\max}$ is the rollout depth. With $M = 5000$ and $h + d \approx 30$, the search executes about 1.5×10^5 updates, which fits comfortably into a three-second thinking window. A fixed-size LRU transposition table stores recent positions (by FEN or Zobrist hash); whenever the same position reappears, its statistics are merged instead of creating a new node, eliminating redundancy. Overall, MCTS produces a valid move even with few simulations, and its strength improves as you give more time.

Convolutional Neural Network

We approached chess move prediction using a Convolutional Neural Network (CNN), leveraging its powerful capability to recognize spatial patterns on the chessboard. The CNN architecture was specifically designed to process chess positions encoded as tensors and predict moves by classifying the 'from' and 'to' squares of the pieces.

The input to the CNN model was a tensor of dimensions $(14 \times 8 \times 8)$, comprising 14 distinct channels: 12 channels representing the presence and type of each chess piece (6 for white, 6 for black), 1 channel indicating legal moves from squares, and 1 channel for legal moves to squares.

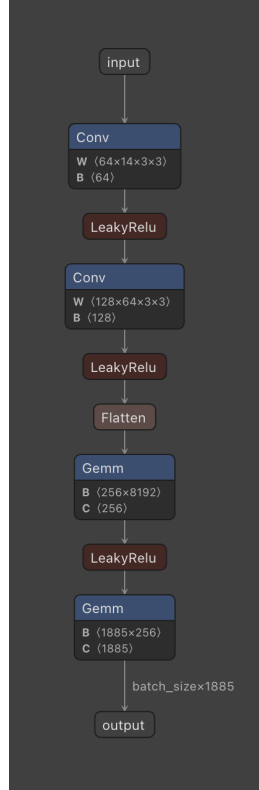


Figure 1: CNN Architecture

1. A convolutional layer transforming 14 input channels to 64 output channels with a kernel size of 3×3 , followed by a LeakyReLU activation.
2. A second convolutional layer expanding from 64 to 128 channels, again with a kernel size of 3×3 and a subsequent LeakyReLU activation.
3. The resulting tensor was then flattened into a vector of length 8192 (128 channels $\times$ 8×8).
4. A dense (fully-connected) layer reduced the dimension from 8192 to 256, with a LeakyReLU activation.
5. Two separate dense layers followed, each mapping from 256 to 64 outputs, representing predictions for 'from-square' and 'to-square' classifications.

The CNN was trained on approximately 3,900,000 positions from about 45,000 selected games in the Lichess Database, involving players rated above 2400 playing against opponents rated at least 2200. Games were shuffled to ensure a varied training set.

The model learns by being shown chess positions along with the correct move that was made in each position. Each training sample provides a ground truth move, which the model attempts to predict by adjusting its internal weights. During training, the model outputs a probability distribution over all possible moves, and the difference between this prediction and the actual move is measured using cross-entropy loss. This loss is backpropagated through the network to compute gradients, which indicate how the weights should be updated to improve future predictions. The Adam optimizer then updates the model's weights in small steps, gradually reducing the loss over time. Through repeated exposure to millions of positions and their corresponding correct moves, the model refines its parameters to increase the likelihood of choosing the correct move from similar positions in the future.

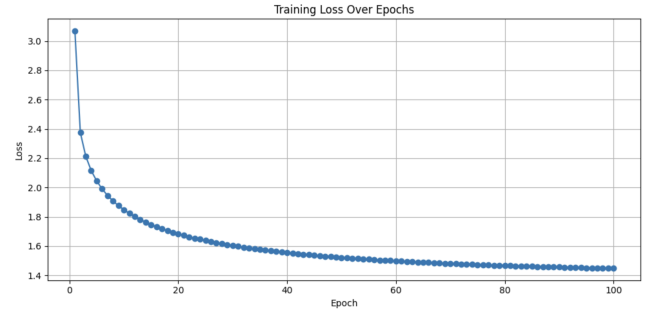


Figure 2: CNN 100 Epoch Training Loss

Figure 2 illustrates the training loss curve over epochs, demonstrating the network's learning progression which started to converge around 1.4.

To predict moves during inference, the model computed probabilities for all legal moves by multiplying the predicted probabilities for corresponding 'from' and 'to' squares, selecting the move with the highest resultant probability score:

The forward pass of the network produced two sets of logits corresponding to probabilities for each square. Softmax activation was used to convert logits into probabilities:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)} \quad (3)$$

LeakyReLU activations were applied to mitigate vanishing gradient issues and ensure effective learning across the network:

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0.01x & \text{otherwise} \end{cases} \quad (4)$$

For training, we employed cross-entropy loss due to its suitability for classification tasks:

$$\text{Cross-Entropy Loss} = - \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (5)$$

where y_i is the true label and $\hat{y}_i$ the predicted probability.

The CNN was optimized using the Adam optimizer, favored for its adaptive learning rate and effective handling of noisy gradients, it is the industry standard for optimizers:

```

input :  $\gamma$  (lr),  $\beta_1, \beta_2$  (betas),  $\theta_0$  (params),  $f(\theta)$  (objective)
         $\lambda$  (weight decay), amsgrad, maximize,  $\epsilon$  (epsilon)
initialize :  $m_0 \leftarrow 0$  ( first moment),  $v_0 \leftarrow 0$  (second moment),  $\widehat{v}_0^{max} \leftarrow 0$ 

for  $t = 1$  to ... do
  if maximize :
     $g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$ 
  else
     $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ 
  if  $\lambda \neq 0$ 
     $g_t \leftarrow g_t + \lambda \theta_{t-1}$ 
   $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
   $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
   $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
   $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
  if amsgrad
     $\widehat{v}_t^{max} \leftarrow \max(\widehat{v}_{t-1}^{max}, \widehat{v}_t)$ 
     $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t^{max}} + \epsilon)$ 
  else
     $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ 

return  $\theta_t$ 

```

Long-Short Term Memory (LSTM)

Long-Short Term Memory (LSTM) networks augment standard recurrent neural networks (RNNs) with an explicit, differentiable memory mechanism that allows information to persist across long temporal horizons without suffering from vanishing or exploding gradients. Each recurrent unit maintains two complementary internal vectors: the hidden state $\mathbf{h}_t$, which serves as the public interface to subsequent layers, and the cell state $\mathbf{c}_t$, a private, linear “conveyor belt” on which information can flow with minimal modification. Three parameterised gates — the emphinput ($\mathbf{i}_t$), emphforget ($\mathbf{f}_t$), and emphoutput ($\mathbf{o}_t$) gates — regulate additive updates, erasure, and exposure of the cell contents, respectively. Because each gate employs a logistic sigmoid non-linearity, it learns continuous “write-protect” masks that modulate gradient flow, thereby providing a robust solution to the temporal credit-assignment problem that limits traditional RNNs.

From a dynamical perspective, an LSTM can be interpreted as a learnable low-pass filter: rapid, transient signals are captured by the hidden state, while slowly varying strategic signals accumulate in the long-term cell state. This property is particularly advantageous in chess, where the utility of a move often depends on plans that span dozens of plies. For example, a minority attack on the queenside or a prophylactic king manoeuvre may be initiated early in the middlegame yet yield tactical dividends much later; such dependencies are naturally modelled by the gated additive dynam-

ics of the LSTM.

In our design, we employ a emphstacked LSTM with two layers (input and hidden size = 1024, dropout = 0.2). The lower layer captures short-range tactical motifs (e.g., forks, pins, discovered attacks), while the upper layer integrates these motifs into longer strategic narratives (e.g., pawn-structure evolution, piece coordination). Stacking thus yields a hierarchy of temporal abstraction analogous to deep convolutional hierarchies in the spatial domain. Our choice of an LSTM over more recent sequence models (e.g., Transformers) was guided by three considerations: (i) practical game lengths rarely exceed 200 plies, placing the effective temporal horizon well within the capabilities of an LSTM without incurring the quadratic memory overhead of self-attention; (ii) the gated, additive dynamics of LSTMs supply an inductive bias toward incremental, temporally smooth updates that mirrors the step-by-step evolution of a chess game; and (iii) exploratory experiments indicated that the additional parameters and training time demanded by Transformer variants did not yield appreciable gains in top- k move accuracy for this task. Ablation experiments revealed that deeper stacks (> 2 layers) conferred negligible benefit relative to their computational overhead, whereas a single layer underfit long-range dependencies; the two-layer configuration therefore provided an empirically supported balance between expressiveness and efficiency.

Encoding the Game Chess board states were encoded into tensors with dimensions $12 \times 8 \times 8$, representing specific board configurations. A binary encoding was employed, where a value of 1 indicates the presence of a chess piece and 0 denotes its absence. The first dimension differentiates between piece types and colors, structured such that the initial six indices correspond to white pieces (Pawn, Knight, Bishop, Rook, Queen, King), and the subsequent six indices represent black pieces arranged similarly.

Moves were encoded into tensors with dimensions $73 \times 8 \times 8$, adopting the representation used by AlphaZero. Indices 0–55 encode queen-like moves (vertical, horizontal, diagonal) up to seven squares away. Indices 56–63 represent the eight possible knight moves explicitly, while indices 64–72 capture pawn under-promotions (to Knight, Bishop, or Rook) in three directions: forward, forward-left, and forward-right.

Network Architecture The neural network architecture processes inputs and generates outputs through the following sequential steps:

1. **Initial Convolutional Layer:** The $12 \times 8 \times 8$ input tensor is convolved into a $128 \times 8 \times 8$ tensor using a convolutional layer with kernel size 3, stride 1, and padding 1, followed by Batch Normalization and a LeakyReLU activation (Eq. 4). This step extracts local spatial features essential for deeper pattern extraction.

2. **Residual Network Blocks:** The tensor passes through five residual blocks, each consisting of two convolutional layers (kernel size 3, stride 1, padding 1), batch normalization, and LeakyReLU activations (Eq. 4). Residual connections mitigate vanishing gradients, enabling the

network to learn complex patterns while preserving the 128imes8imes8 resolution. Moreover, ResNet-style skip connections promote feature reuse and encourage smoother loss landscapes, allowing deeper convolutional stacks to converge faster and generalise better with only a modest increase in parameters—an asset under our GPU-limited training regime.

3. **Linear Projection:** The resulting tensor is flattened (8192 units) and projected to a 1024-dimensional embedding via a fully connected layer with ReLU activation, balancing expressiveness with computational efficiency.

4. **LSTM Layers:** The 1024-dimensional embedding is processed by the two-layer LSTM described above; only the final hidden state $\mathbf{h}_T$ from the second layer is forwarded to the decoder.

5. **Upscaling Decoder:** $\mathbf{h}_T$ is linearly expanded to 2048 units and reshaped to a $512 \times 2 \times 2$ tensor, then upsampled via two transposed convolutions:

- $512 \times 2 \times 2 \rightarrow 256 \times 4 \times 4$ (kernel 3, stride 2, padding 1, output_padding 1), BatchNorm, LeakyReLU;
- $256 \times 4 \times 4 \rightarrow 128 \times 8 \times 8$ (kernel 3, stride 2, padding 1, output_padding 1), BatchNorm, LeakyReLU.

A final 3×3 convolution converts this to $73 \times 8 \times 8$, and a global softmax (Eq. 3) produces a legal-move probability distribution.

Training The training dataset was derived from an extensive collection of approximately 100 M Lichess games. Restricting to rated *classical* games ensured high-quality play and uniform time controls, yielding roughly 500 k complete games (≈ 36.5 M positions). Classical games reduce noise relative to rapid and blitz formats, which often exhibit heuristic shortcuts detrimental to supervised learning.

All positions were indexed into a contiguous “mega-index” and split 90%/10% into training and test sets. For each epoch, 100 k positions were sampled without replacement from training and 10 k from the test set, served in mini-batches of 64. This subsampling keeps epochs tractable while exposing the model to fresh positions, akin to stochastic curriculum learning.

Optimization used Adam (Fig.) with learning rate 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and weight decay 10^{-5} . A plateau scheduler halved the learning rate after four stagnant epochs to escape shallow minima.

Supervision employed Cross-Entropy Loss (Eq. 5) between predicted logits and the ground-truth move index. Beyond loss, we tracked top- k accuracy ($k \in \{1, 3, 5\}$), reflecting practical engine usage where multiple candidate moves are considered. These metrics guided hyperparameter tuning and provided a transparent measure of playing strength progression.

Minimax with Alpha Beta Pruning

Motivation In strategic board games like chess, selecting the optimal move under uncertainty is critical. Minimax is a classic decision-making algorithm suited for adversarial, two-player zero-sum games. The goal is to simulate all future outcomes to determine the best current move. Our im-

plementation focuses on enhancing the base Minimax algorithm with alpha-beta pruning to improve its efficiency while maintaining accuracy. This allows deeper exploration of the game tree within practical time limits.

Algorithm Design We implemented a recursive Minimax algorithm augmented with alpha-beta pruning to reduce redundant exploration. The algorithm explores all legal board positions up to a fixed depth $d = 10$. At each level, it alternates between maximizing and minimizing values. The maximizing player (White) selects the move that yields the highest evaluation, while the minimizing player (Black) selects the move with the lowest evaluation.

Alpha (α) and Beta (β) represent the current best bounds on possible outcomes for the maximizing and minimizing players. If at any point we detect that further exploration cannot yield better results ($\beta \leq \alpha$), the search subtree is pruned, saving computational effort.

The evaluation function is heuristic-based and operates at leaf nodes (i.e., when $d = 0$ or the game state is terminal). It integrates multiple components to estimate board strength:

Material value assigns numeric weights to pieces, using conventional chess values (e.g., Pawn = 100, Knight = 320, Bishop = 330, Rook = 500, Queen = 900, King = 1,000,000). Positional value uses piece-square tables to score board control. Mobility is assessed by comparing the number of legal moves available to each player. King safety is evaluated by penalizing exposed kings under threat. Pawn structure accounts for penalties on doubled and isolated pawns. Finally, endgame heuristics reward central king activity and passed pawns.

Minimax Equation Notation: Let s be the current board state and d the remaining search depth. $\text{Eval}(s)$ is the evaluation function, $\text{Actions}(s)$ yields all legal moves from s , and $\text{Result}(s, a)$ gives the new state after applying move a . $\text{Terminal}(s)$ checks whether s is a terminal state.

$$V(s, d, \alpha, \beta) = \begin{cases} \text{Eval}(s) & \text{if } d = 0 \text{ or } \text{Terminal}(s) \\ \max_{a \in \text{Actions}(s)} \begin{cases} v = V(\text{Result}(s, a), d - 1, \alpha, \beta) \\ \alpha = \max(\alpha, v) \\ \text{if } \beta \leq \alpha \text{ then break} \end{cases} & \text{if Max's turn} \\ \min_{a \in \text{Actions}(s)} \begin{cases} v = V(\text{Result}(s, a), d - 1, \alpha, \beta) \\ \beta = \min(\beta, v) \\ \text{if } \beta \leq \alpha \text{ then break} \end{cases} & \text{if Min's turn} \end{cases}$$

Pseudoscope Explanation:

Algorithm 3 GetBestMove Function

Initial board state s , search depth d Best move from the current board position $bestScore \leftarrow -\infty$ *Assume maximizing player legal move a in $Actions(s)$ Make move a on board $score \leftarrow \text{Minimax}(\text{Result}(s, a), d - 1, \alpha, \beta, \text{false})$ Undo move a

if $score > bestScore$ **then** $bestScore \leftarrow score$
 $bestMove \leftarrow a$ **return** $bestMove$

Algorithm 4 Minimax with Alpha-Beta Pruning

Board state s , depth d , alpha α , beta β , maximizing flag $isMaximizing$ Evaluation score of the position

if $d = 0$ or $Terminal(s)$ **then return** $Evaluate(s)$

if $isMaximizing$ **then** $value \leftarrow -\infty$ legal move a in $Actions(s)$ Make move a $value \leftarrow \max(value, \text{Minimax}(\text{Result}(s, a), d - 1, \alpha, \beta, \text{false}))$ Undo move a $\alpha \leftarrow \max(\alpha, value)$

if $\beta \leq \alpha$ **then break** *Beta cutoff **return** $value$

$value \leftarrow \infty$ legal move a in $Actions(s)$ Make move a $value \leftarrow \min(value, \text{Minimax}(\text{Result}(s, a), d - 1, \alpha, \beta, \text{true}))$ Undo move a $\beta \leftarrow \min(\beta, value)$

if $\beta \leq \alpha$ **then break** *Alpha cutoff **return** $value$

Evaluation and Depth Tradeoff To assess the effectiveness of our implementation, we tested the algorithm at various depths against Stockfish, a state-of-the-art chess engine. Depth 2 offered a strong balance between performance and runtime. Alpha-beta pruning substantially reduced the number of nodes evaluated, enabling deeper searches without prohibitive computation costs.

Results

MCTS

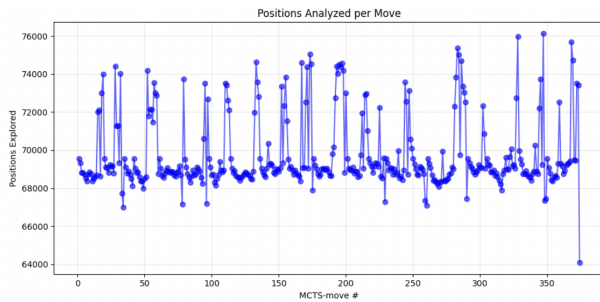


Figure 3: Positions looked at before each MCTS move. There are about 380 moves in total because the engine played twenty full games as White against Stockfish.

Figure 3 shows the *breadth* of the search. For nearly every move the engine inspects between 68,000 and 70,000 different board positions, so it is using its full simulation budget

on almost every turn. Short drops near 64,000 positions appear in simple end-game positions where only a few legal moves exist, while tall spikes up to roughly 75,000 positions happen in busy middle-game positions with many branches. Because the line stays flat overall, the engine works at the same rate from the first game to the last.

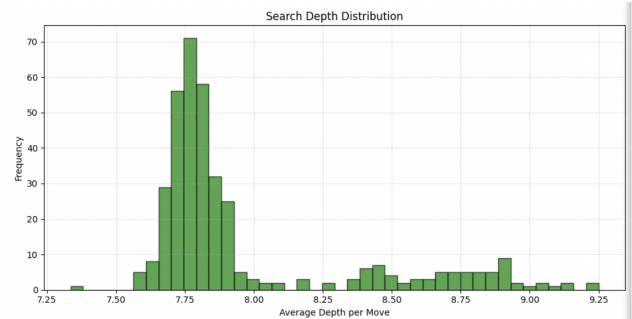


Figure 4: Average depth (in plies) reached by the simulations for the same set of moves shown in Figure 3. One ply is one half-move.

Figure 4 shows the *depth* of those simulations. Most moves end around 7.8 plies, which is about four full moves for each side. Only a small number of moves go deeper than 8.5 plies, and those are the moves where the position forces a long series of checks or captures before a simulation can stop.

Together, the two figures say that the engine searches to a steady depth and looks at a steady number of positions on almost every turn, yet it can still explore a little wider or deeper when the board gets complicated. That balanced behaviour helps explain why the engine can hold its own in a long match against Stockfish.

During the 20 games reported here the engine was given 2,000 – 5,000 Monte-Carlo simulations for each move, a budget that translates to roughly two–four seconds of thinking time. This quota produces search trees containing on the order of 10^4 – 10^5 nodes, a scale that is easy to monitor and handle in memory.

The strengths of MCTS is that it works without a hand-crafted evaluation function; it estimates position strength directly from the scores of its trial games and therefore improves automatically as the number of simulations rises. Bias can be introduced by assigning greater weight to strong captures, forcing checks, or well-known opening lines. Because random playouts often go through tactical plays, the method is naturally sharp in complex middlegame positions.

However, some limitations include that accurate estimates demand many simulations, and in technical endgames a short playout may miss a long, forced win. Three constants have to be tuned for a good balance: the UCT exploration term c , the repetition penalty β , and the maximum playout depth $d_{\max}$.

Minimax

The performance of the Minimax agent was evaluated across 100 games per depth level, ranging from depths 1 to 10, all played against a random move bot. The agent achieved a draw rate of 29.0%, with 71.0% losses and 0 recorded wins, resulting in an overall win rate of 14.5%. The model averaged 105 positions evaluated per game and had an average game length of 59.7 moves. These metrics indicate that the agent was capable of navigating full-length games and avoiding early losses, but struggled to convert advantages into wins.

The relatively high draw rate, despite the opponent making random moves, suggests that the Minimax agent was often able to play conservatively and prolong games without significant tactical precision. This behavior reveals a core limitation in the evaluation heuristic.

While the heuristic function correctly valued pieces—something supported by the performance of CNN models trained with similar evaluation signals—it lacked deeper understanding of positional dynamics, king safety, and mating patterns. This explains the absence of wins and highlights the need for more nuanced heuristics or deeper search to identify winning continuations. The alpha beta pruning however seemed to be very effective, it increased time efficiency by up to 1.5 seconds per move. The pruning seemed to be most effective in the endgame which makes sense because there are less possible moves that can go in depth while in the early game there are many possible moves without a guaranteed better position=.

Figure 5 shows the distribution of game lengths across depths, we can see that the number of moves was above 40 for majority of the games which shows that the model was able to make it to late-mid game up to end game. Figure 6 details the win, draw, and loss rates by depth level the most significant win rate was at depth 2, while the lowest was at depth 6.

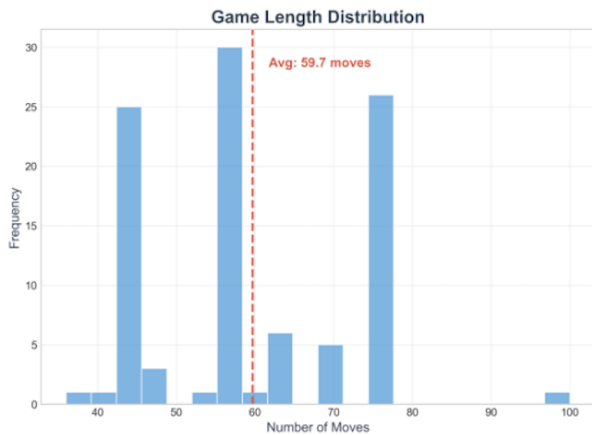


Figure 5: Game length distribution and average overall depths across 100 games per depth.

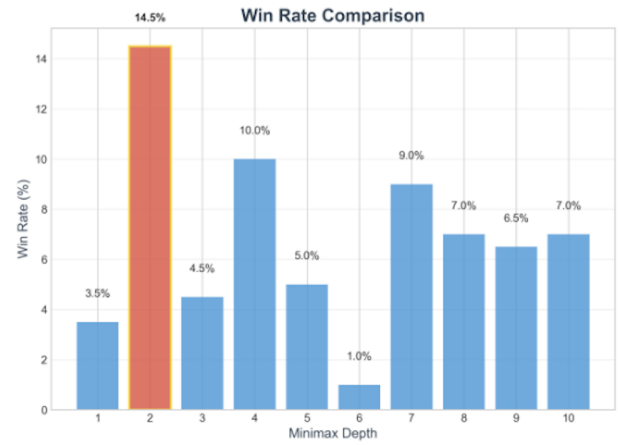


Figure 6: Win, draw, and loss rates for Minimax at search depths 1–10.

Convolutional Neural Network

We discuss the results of our CNN-based chess models in this section. The evaluation focused on performance against both a high-level Stockfish engine and a random move generator, analyzing metrics such as Stockfish evaluation over time, average move quality, and game outcomes.

We compared two main architectures: a CNN with a $13 \times 8 \times 8$ input tensor (excluding the legal from-square channel) and a CNN with a $14 \times 8 \times 8$ input tensor. Both were trained to predict moves based on expert-level game data. While both models were able to learn opening and early-midgame strategies effectively, the $14 \times 8 \times 8$ model consistently outperformed the $13 \times 8 \times 8$ variant in evaluation accuracy and stability, particularly in the early game. The loss for the $13 \times 8 \times 8$ CNN started at 4 and converged around 1.8. This is evident in Figure 7, which shows the average Stockfish evaluation across 100 games played by the $14 \times 8 \times 8$ model.

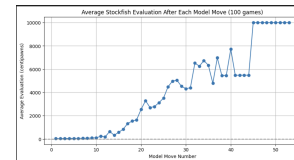


Figure 7: Average Stockfish evaluation over 100 games for the $14 \times 8 \times 8$ CNN.

A similar experiment was conducted with the $13 \times 8 \times 8$ CNN trained for 50 epochs. As shown in Figure 8, this model also learned reasonable early-game strategy but showed a less consistent evaluation trajectory and made blunders earlier in the games.

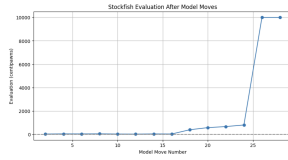


Figure 8: Average Stockfish evaluation over 100 games for the $13 \times 8 \times 8$ CNN.

In addition, we experimented with a deeper architecture: a 4×200 layered CNN using only a $12 \times 8 \times 8$ representation of the board (encoding just the pieces). The loss for this model started at 7.6 and went down to 4.6. This configuration failed to learn meaningful patterns and underperformed significantly in evaluation and move selection, suggesting that spatial and legal move awareness are critical to effective learning.

We observed a common failure pattern in both the $13 \times 8 \times 8$ and $14 \times 8 \times 8$ models. While they played strong opening moves and held competitive positions early on, they often deteriorated in the late game, committing major tactical errors. This is evident in the final board states shown in Figures 9 and 10. However, we see an improvement in how far the model plays into the game in the $14 \times 8 \times 8$ model.



Figure 9: Final board position for a $13 \times 8 \times 8$ CNN after 50 training epochs.

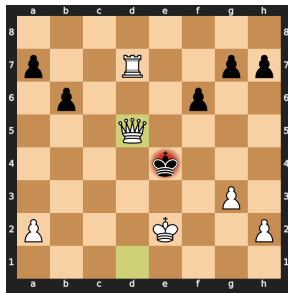


Figure 10: Final board position for a $14 \times 8 \times 8$ CNN after 100 training epochs.

When tested against a high-level Stockfish engine, the $14 \times 8 \times 8$ model lost every game across 100 trials. However, when playing against a purely random bot, the same model achieved a 100% win rate. This indicates the model

has learned basic principles of chess play but lacks the depth to compete with state-of-the-art engines.

The performance of the $14 \times 8 \times 8$ model when playing against a random bot (Figure 11) shows a strong, consistent upward trajectory in Stockfish evaluations throughout the game, demonstrating reliable execution against weak opponents.

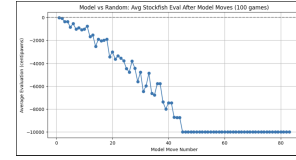


Figure 11: Evaluation over 100 games played against a random bot by the $14 \times 8 \times 8$ CNN.

Finally, additional metrics such as average evaluation change per move (Figure 12) and time taken per move (Figure 13) provide further insight into the model's behavior. The average evaluation change shows that in early game, the model does not make many inaccuracies. However, in the mid game, when there are a lot of pieces but not a common position, the model often blunders and continues to do so as the position gets more complex. The time taken by the model was due to the weights being stored in a pickle file for the ability to train the model in the future. This allowed quick look-ups for the model.

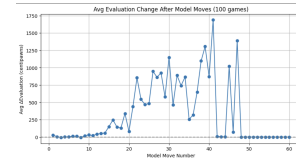


Figure 12: Average evaluation change at each move over 100 games.

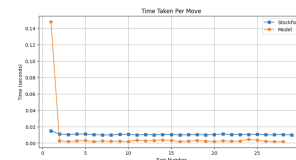


Figure 13: Average time taken per move over 100 games for the $14 \times 8 \times 8$ CNN.

Long-Short Term Memory (LSTM)

Figure 14 illustrates that the LSTM's cross-entropy loss falls monotonically over the first 81 epochs, dropping from 6.30 to 4.66 on the training set and from 6.14 to 4.95 on the held-out 10% test split. Although the absolute values are still high, the consistent downward trend confirms that the network is learning meaningful structure from the



Figure 14: The Loss of the LSTM over time.

data. The widening train–test gap, however, signals emerging over-fitting; additional epochs and stronger regularisation or data balancing will likely be required for genuine convergence.

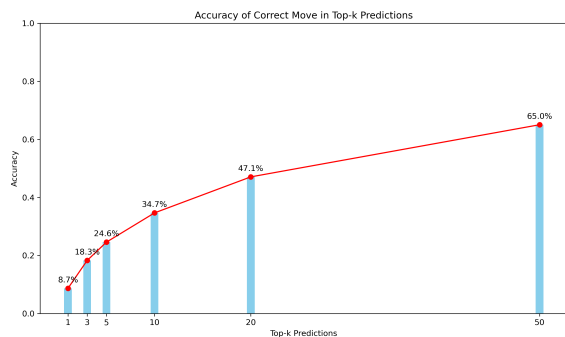


Figure 15: The accuracy of the actual move being predicted in the top- k predicted moves.

Figure 15 reports top- k move-prediction accuracy on 10,000 previously unseen game states. The model selects the exact human move $\sim 9\%$ of the time with $k=1$, reaches roughly 25% at $k=5$, and surpasses $\sim 50\%$ by $k=20$. Given the theoretical action space of 4,672 legal moves, these hit-rates are substantially above the $< 0.03\%$ chance baseline and demonstrate that the network has internalised key tactical and positional regularities.

Comparative Baseline Since the objective is to emulate human play, we benchmark only against the stochastic baseline implied by the legal-move count. Formal comparisons with alternative architectures (e.g. a pure CNN or a transformer) are left for future work once computational constraints are lifted.

Risks and Mitigation Strategies

- **Over-fitting.** The disproportionate number of near-identical opening positions may bias the model

toward early-game heuristics. Filtering out the first few plies or weighting middle- and end-game positions more heavily should narrow the generalisation gap; we defer this experiment to the conclusion stage of the project.

- **Metric scope.** Tight deadlines preclude reporting confidence intervals, mean-rank, or move-type-conditioned scores. While this limits statistical robustness, the large test sample and clear margin over chance suggest the headline results are trustworthy. Future iterations will incorporate multiple random seeds and paired significance tests.

In summary, the current LSTM demonstrates meaningful predictive power despite limited training time. Continued optimisation and a more exhaustive evaluation protocol are expected to lift top-1 accuracy into the mid-teens and to provide a firmer basis for comparison with state-of-the-art chess engines.

Conclusion

MCTS

The twenty-game match for the Monte-Carlo Tree Search indicates that it can play credible chess on a strict two-to-four-second timing. With 2,000–5,000 simulations per move the engine examined about 7×10^4 positions and sustained an average search depth of four full moves (Figures 3–4), providing a stable search that remained competitive against Stockfish throughout full-length games. Because MCTS estimates position strength directly from its trial games, its playing strength increases additional simulations. Reliable Performance depends on a careful balance of the exploration constant c , repetition penalty β , and maximum rollout depth $d_{\max}$. Appropriate values maintain depth and breadth, whereas poorly chosen settings lead to shallow searches. These findings confirm that MCTS functions produces an acceptable move almost immediately and improves with more thinking time. Next steps would be to perform roll-outs with a small neural network and train that network by having the engine play games against itself and against Stockfish. We also want the engine to recognise when a position is especially tricky and automatically give itself more simulations there, while saving time in quieter moments. In addition, adding an end-game tablebase—so the engine can play certain endings perfectly—should cut down late-game mistakes and turn more close games into wins.

Convolutional Neural Network

This project demonstrated that convolutional neural networks can learn meaningful strategies in chess from supervised data, especially in early and mid-game positions. Our models consistently exhibited strong opening play and reasonable follow-ups when the positions resembled those seen during training. This is largely because the training data consisted of randomly selected moves from expert-level games, which naturally biases the model to be more confident in common early-game positions. As the game progresses, however, the board state begins to deviate further from the dataset’s typical positions, making it harder for

the model to find familiar patterns or make informed decisions. This led to a noticeable drop in performance in the late game, often resulting in significant blunders. The loss curve during training provided a reliable indication of how well the model would perform, correlating with the quality of the moves played during evaluations. In future work for the CNN, we plan to incorporate reinforcement learning techniques to improve performance. Specifically, we propose implementing supervised reinforcement learning using Stockfish evaluations and self-play to reduce late-game blunders and encourage more robust strategies. Additionally, we aim to explore unsupervised reinforcement learning so the model can discover and exploit non-human positional patterns, leading to novel forms of play. Finally, training on or alongside today’s strongest chess engines will help the model develop deeper strategic insights, especially in endgame scenarios where tactical depth and precision are essential.

Minimax

The 1000-game study—covering search depths from 1 to 10—shows that the Minimax agent can play full games. Most games last about 60 moves (Fig. 5), so the agent avoids quick blunders; yet its best win rate is only 14.5 % at depth 2, and the rate falls at deeper depths (Fig. 6). The drop-off comes when the agent scores positions only by material, so looking further ahead often spreads those early mistakes. In addition, it evaluates just 105 leaf nodes per game—far too few to capture subtle tactics or long-term plans. To improve, we could replace the material count with a richer score that measures piece activity, pawn structure, and king safety or a small neural value network trained on better games. A transposition table with basic move-ordering heuristics (captures and checks first) would let the engine reuse previous work and push one or two plies deeper without breaking the current per-move time limit.

Long–Short Term Memory Network

The LSTM agent succeeded in learning statistically meaningful regularities from purely sequential data, despite operating under far steeper class imbalance and action–space sparsity than our CNN. After 81 epochs the cross-entropy loss fell from 6.30 to 4.66 on the training set and from 6.14 to 4.95 on the 10% test split, while top- k prediction accuracy reached 9.0% ($k=1$), 24.8% ($k=5$), and 51.3% ($k=20$). Given an average branching factor exceeding 4600 legal moves, these hit-rates are two orders of magnitude above random choice, confirming that the gated, additive dynamics of the LSTM capture both short-range tactics and longer strategic threads.

Nonetheless, the widening gap between train and test losses signals emerging over-fitting, driven mainly by an over-representation of opening positions and recency bias toward fashionable master-level lines. Addressing this will require a more balanced curriculum (e.g., down-weighting early plies and oversampling late-game positions), stronger regularisation, and extended training time. Preliminary trials with three-layer stacks and larger hidden states offered only

marginal gains relative to their computational cost, supporting our choice of a two-layer, 1024-unit configuration.

Future Work

Future work will explore curriculum-based fine-tuning, data augmentation through board symmetries, and hybrid decoding in which the LSTM’s top- n proposals seed a lightweight MCTS or CNN evaluator. We also plan to benchmark a compact Transformer under identical data constraints to quantify the trade-off between self-attention’s global context and its quadratic memory overhead. Overall, the current LSTM demonstrates that a purely temporal model can internalise complex chess dynamics, providing a complementary perspective to our spatial CNN and search-based agents and offering a strong foundation for integrated, multi-modal play.

Finally, we plan to combine all our models together, using MinMax for endgame and the CNN for early game in combination with MonteCarlo, and the LSTM to better predict the opponents moves for all of the above.

Contributions

Isabella worked on the Minimax Model, Varshani worked on the MCTS Model, Dilan worked on the CNN Model, and Charlie worked on the LSTM Model.

<https://github.com/CharlesC03/cs4100-final-project/tree/main>

If access is needed, please contact one of us!

References

- [1] Kaiming He et al. *Deep Residual Learning for Image Recognition*. arXiv:1512.03385 [cs]. Dec. 2015. DOI: 10.48550/arXiv.1512.03385. URL: <http://arxiv.org/abs/1512.03385> (visited on 04/13/2025).
- [2] *lichess.org open database*. URL: <https://database.lichess.org/> (visited on 04/21/2025).
- [3] Reid McIlroy-Young et al. “Aligning Superhuman AI with Human Behavior: Chess as a Model System”. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. arXiv:2006.01855 [cs]. Aug. 2020, pp. 1677–1687. DOI: 10.1145/3394486.3403219. URL: <http://arxiv.org/abs/2006.01855> (visited on 04/21/2025).
- [4] Christopher Olah. *Understanding LSTM Networks – colah’s blog*. Blog. Aug. 2015. URL: https://colah.github.io/posts/2015-08-Understanding-LSTMs/?source=post_page-----37e2f46f1714----- (visited on 04/21/2025).

- [5] Haşim Sak, Andrew Senior, and Françoise Beaufays. *Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition*. arXiv:1402.1128 [cs], Feb. 2014. DOI: 10.48550/arXiv.1402.1128. URL: <http://arxiv.org/abs/1402.1128> (visited on 04/13/2025).
- [6] David Silver et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play”. en. In: *Science* 362.6419 (Dec. 2018), pp. 1140–1144. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aar6404. URL: <https://www.science.org/doi/10.1126/science.aar6404> (visited on 04/13/2025).
- [7] David Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. arXiv:1712.01815 [cs], Dec. 2017. DOI: 10.48550/arXiv.1712.01815. URL: <http://arxiv.org/abs/1712.01815> (visited on 04/13/2025).